

دليل ربط Flutter مع Backend

ماذا يحتوي هذا التسليم

- backend مع هذا ال Flutter جاهز لربط Starter وهو `flutter\_integration\_kit/`
- ملف PDF باللغة الإنجليزية.
- ملف PDF باللغة العربية.

العقد الحالي لل backend

- مسار ال API الأساسي هو `api/v1/`
- منفذ التشغيل المحلي في هذا المشروع هو `3000`
- ملف OpenAPI موجود على `api/v1/docs/openapi.json/`
- تسجيل الدخول يعيد `accessToken` داخل جسم الاستجابة
- التجديد وتسجيل الخروج يعتمدان على Cookie اسمها `refreshToken`
- اسم حقل رفع الصورة الشخصية هو `avatar`
- اسم حقل رفع صور المنتج هو `images`
- روابط الصور ترجع بشكل نسبي مثل `.../uploads/products/`

الخطوة 1: شغل ال backend أولاً

استخدم هذا المشروع كمرجع أساسي أثناء الربط.

```
npm run dev
```

تأكد من هذه الروابط:

- واجهة التوثيق: `http://localhost:3000/api/v1/docs`
- ملف OpenAPI الخام: `http://localhost:3000/api/v1/docs/openapi.json`

الخطوة 2: افتح مشروع Flutter الخاص بالموبايل

لا يحتوي على مشروع Flutter نفسه، لذلك المطلوب هو نقل ال starter إلى مشروع Flutter منفصل.
هذا المستودع

إذا كان لديك مشروع Flutter موجود بالفعل:

محتويات `flutter\_integration\_kit/lib/` إلى مجلد `lib/` داخل مشروع Flutter أو ضمها كطبقة مستقلة.
1. انسخ

2. أضيف Dependencies الموجودة في `flutter\_integration\_kit/pubspec.yaml`.

3. شغل `flutter pub get`.

إذا كنت تبدأ من الصفر:

```
flutter create ai_rent_mobile
cd ai_rent_mobile
flutter pub add dio dio_cookie_manager cookie_jar path path_provider flutter_secure_storage
```

ثم انسخ ملفات ال starter إلى المشروع.

الخطوة 3: اضبط عنوان ال API الصحيح

استخدم `dart-define--` حتى يمكن تغيير عنوان ال backend حسب البيئة.

القيم المقترحة:

- Android emulator: `http://10.0.2.2:3000/api/v1`
- iOS simulator: `http://127.0.0.1:3000/api/v1`
- جهاز حقيقي على نفس الشبكة: `http://YOUR\_LOCAL\_IP:3000/api/v1`

مثال:

```
flutter run --dart-define=API_BASE_URL=http://10.0.2.2:3000/api/v1
```

ال starter يقرأ هذه القيمة من `AppConfig.fromEnvironment()`.

الخطوة 4: اسمح باتصال HTTP أثناء التطوير

إذا كنت تختبر محلياً عبر HTTP وليس HTTPS فالموبايل يحتاج إعدادات إضافية.

Android:

في ملف `android/app/src/main/AndroidManifest.xml` أضيف:

```
<application
  android:label="ai_rent_mobile"
  android:usesCleartextTraffic="true">
</application>
```

iOS:

في ملف `ios/Runner/Info.plist` أضيف:

```
<key>NSAppTransportSecurity</key>
<dict>
  <key>NSAllowsArbitraryLoads</key>
  <true/>
```

</dict>

استخدم هذا فقط أثناء التطوير المحلي ثم انتقل إلى HTTPS في البيئات الفعلية.

الخطوة 5: شغل الـ bootstrap في `main.dart`

الـ starter يحتوي بالفعل على تهيئة واضحة:

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final dependencies = await AppDependencies.bootstrap();
  runApp(IntegrationStarterApp(dependencies: dependencies));
}
```

هذه التهيئة تبني:

- `AppConfig`
- `SessionStore`
- `CookieJar` دائم
- `ApiClient`
- backend منفصل لكل جزء من الـ Repository

الخطوة 6: اربط المصادقة بالشكل الصحيح

هذا الـ backend يستخدم طريقتين معا:

1. `POST /auth/login` يعيد Access Token داخل الاستجابة
2. نفس العملية تحفظ Refresh Token داخل Cookie
3. الطلبات المحمية تستخدم `Authorization: Bearer <token>`
4. عند انتهاء الـ access token يتم استدعاء `auth/refresh-token/` باستخدام الـ cookie المحفوظة

الـ starter يعالج هذا من خلال:

- `SessionStore` لحفظ access token
- `PersistCookieJar` لحفظ refresh cookie
- `ApiClient` interceptor لإعادة المحمية القابلة للإعادة

مثال تسجيل دخول:

```
await dependencies.auth.login(
  email: emailController.text.trim(),
  password: passwordController.text,
);

final profile = await dependencies.users.getMyProfile();
```

الخطوة 7: ابدأ بالشاشات العامة أولاً

ابدأ بالنقاط التي لا تحتاج تسجيل دخول حتى تتأكد أن الربط يعمل بسرعة.

أمثلة:

```
final categories = await dependencies.categories.listCategories();

final products = await dependencies.products.listProducts(
  page: 1,
  limit: 10,
  search: 'camera',
);
```

الترتيب المقترح:

1. قائمة التصنيفات
2. قائمة المنتجات العامة
3. تفاصيل المنتج
4. الملف العام للمالك

الخطوة 8: اربط مسارات المستأجر والمالك

بعد نجاح تسجيل الدخول اربط الأجزاء المحمية بهذا الترتيب:

1. الملف الشخصي
2. المفضلة
3. فحص التوفر
4. إنشاء طلب إيجار
5. حجوزاتي وطلباتي
6. الإشعارات
7. التقييمات

مثال فحص التوفر:

```
final result = await dependencies.rentals.checkAvailability(
  productId: productId,
  startDate: startDate,
  endDate: endDate,
  rentalPeriodType: 'daily',
  quantity: 1,
);
```

مثال إنشاء طلب إيجار:

```
await dependencies.rentals.createRental(
  productId: productId,
  startDate: startDate,
  endDate: endDate,
  rentalPeriodType: 'daily',
  quantity: 1,
  renterNotes: 'Please confirm pickup time.',
);
```

الخطوة 9: اربط رفع الملفات بالطريقة الصحيحة

يوجد نقطتان تستخدمان Multipart Form Data:

- `POST /users/upload-avatar`
- `POST /products/{id}/images`

أسماء الحقول الصحيحة موجودة بالفعل في ال starter:

- صورة المستخدم: `avatar`
- صور المنتج: `images`

أمثلة:

```
await dependencies.users.uploadAvatar(filePath);

await dependencies.products.uploadProductImages(
  productId: productId,
  filePaths: selectedImagePaths,
);
```

ملاحظات مهمة:

- الحد الأقصى للصورة الشخصية 2 MB
- الحد الأقصى لكل صورة منتج 5 MB
- الحد الأقصى لعدد صور المنتج 10

الخطوة 10: حول روابط الصور إلى روابط كاملة

ال backend يعيد روابط نسبية مثل:

```
/uploads/avatars/example.jpg
/uploads/products/example.jpg
```

داخل Flutter حولها إلى رابط كامل قبل عرض الصورة:

```
final imageUrl = dependencies.config.resolveServerPath(rawPathFromApi);
```

الخطوة 11: اربط الإشعارات وحالة غير المقروء

نقاط الإشعارات هي:

- `GET /notifications`
- `GET /notifications/unread-count`
- `PUT /notifications/{id}/read`
- `PUT /notifications/read-all`

التدفق المقترح:

1. تحميل عدد غير المقروء بعد فتح التطبيق أو بعد تسجيل الدخول
2. فتح شاشة الإشعارات
3. تعليم الإشعار كمقروء عند فتحه
4. إعادة تحميل الشارة الخاصة بعدد غير المقروء

مثال:

```
await dependencies.notifications.markAsRead(notificationId);
final unread = await dependencies.notifications.getUnreadCount();
```

الخطوة 12: اربط صفحات الإدارة فقط لمستخدم admin

كل مسارات الإدارة تبدأ بـ `admin/` وتحتاج:

- صالح access token
- role `admin` يساوي role

الترتيب المقترح لربط الإدارة:

1. لوحة الإحصاءات
2. المستخدمون
3. مراجعة المنتجات
4. الإجراءات
5. التقارير

مثال:

```
final dashboard = await dependencies.admin.getDashboard();
final pendingProducts = await dependencies.admin.getProducts(
  isApproved: false,
  page: 1,
  limit: 20,
);
```

الخطوة 13: هيكل المجلدات المقترح داخل Flutter

حافظ على فصل المسؤوليات الموجود في ال starter:

```
lib/  
  src/  
    config/  
    core/  
      http/  
      storage/  
      models/  
    features/  
      auth/  
      users/  
      categories/  
      products/  
      rentals/  
      reviews/  
      wishlist/  
      recommendations/  
      behavior/  
      notifications/  
      admin/
```

هذا يجعل المشروع أسهل في الفهم والصيانة والتتبع.

الخطوة 14: قائمة التحقق النهائية

نفذ هذه الاختبارات بالترتيب:

1. التطبيق يقرأ التصنيفات من ال backend
2. تسجيل الدخول يحفظ access token وال refresh cookie
3. يعمل بعد تسجيل الدخول `/users/me`.
4. قائمة المنتجات العامة وتفاصيل المنتج تعمل
5. روابط الصور النسبية تتحول إلى روابط كاملة وتظهر بشكل صحيح
6. الإضافة إلى المفضلة والحذف من المفضلة يعملان
7. فحص التوفر وإنشاء الإيجار يعملان
8. عدد الإشعارات غير المقروءة ينخفض بعد تعليم الإشعار كمقروء
9. مسارات الإدارة تعمل فقط مع حساب admin
10. تسجيل الخروج يمسح التوكن المحلي والكوكي

مجموعات ال endpoints التي يغطيها ال starter

- المصادقة
- المستخدم

- التصنيفات
- المنتجات
- الإيجارات
- التقييمات
- المفضلة
- التوصيات
- تتبع السلوك
- الإشعارات
- الإدارة

ملاحظة أخيرة

تنفيذي خطوة بخطوة. انسخ ال starter إلى مشروع الموبايل ثم اربط كل شاشة مع ال repository المناسب. غير موجود داخل هذا المستودع، فالتسليم هنا عبارة عن Starter متوافق مع ال backend بالإضافة إلى دليل لأن مشروع Flutter نفسه